



**Department of Computer Science
Air University Aerospace and Aviation Campus,
Kamra**

Project Report

**Cowrie Honeypot Deployment & Wazuh SIEM
Integration**

Prepared by:

Addan Shahbaz
Hamza Raiz
Alija Mehdi
Jalisha Eman
Hadiya Zuberi

Submitted to:

Ma'am Mahnoor Gillani

1. EXECUTIVE SUMMARY

This report documents the end-to-end deployment of a Cowrie SSH Honeypot integrated with the Wazuh Security Information and Event Management (SIEM) platform. The project was executed in a controlled virtual lab environment to simulate real-world attacker behavior and validate automated threat detection capabilities.

Cowrie is an open-source SSH and Telnet honeypot designed to trap, record, and analyze unauthorized access attempts. When an attacker connects, Cowrie accepts any credentials and presents a convincing simulated shell — all without exposing any real systems. Every command, keystroke, and credential attempt is logged silently in the background.

Wazuh acts as the centralized SIEM that ingests, parses, and triggers alerts from Cowrie's logs in real time. Custom decoders and rules were written to map Cowrie events to Wazuh's alerting engine, and the MITRE ATT&CK framework was leveraged to automatically classify attacker techniques.



Key outcomes of this project:

- Successfully deployed Cowrie honeypot on a dedicated Ubuntu server, listening on ports 22 and 2222.
- Simulated a realistic attacker using Kali Linux — NMAP scanning, followed by SSH brute-force attempts.
- Integrated Cowrie with Wazuh via a custom decoder and rule set, enabling real-time alert generation.
- MITRE ATT&CK technique T1110.001 (Password Guessing) was automatically identified and tagged in Wazuh.
- All 6 Jira project tasks were completed with Medium priority, achieving 100% done status.

2. TOOLS & ENVIRONMENT

2.1 Network Topology

 Honeypot Server (Ubuntu) IP: 192.168.184.134 Ports: 22 (SSH real) 2222 (Cowrie)	 Attacker Machine (Kali Linux) IP: 192.168.184.129 Tools: NMAP, SSH client
--	--

The honeypot server (192.168.184.134) hosted the Cowrie service on port 2222. The attacker machine (192.168.184.129) performed all scanning and connection attempts against this IP. Both machines operated within the same VMware virtual network segment.

2.2 Technology Stack

Tool / Technology	Role in Project	Version / Notes
Cowrie Honeypot	Fake SSH server to trap attackers	v2.6.1 (open-source)
Wazuh SIEM	Security monitoring & alerting platform	v4.12.0
Ubuntu Linux	Honeypot host machine	Server edition
Kali Linux	Attacker simulation machine	Rolling release
NMAP	Network port scanner used by attacker	v7.95
OpenSSH	SSH service prerequisite for Cowrie	OpenSSH_10.0p2
Python 3	Cowrie runtime environment	v3.13.3
Jira	Project tracking & task management	Cloud edition

3. PROJECT TIMELINE (Jira)

All project tasks were managed in Jira under the epic HWI-1: Cowrie Honeypot Deployment & Wazuh Integration. The timeline below shows all six child tasks, each completed during the project sprint.

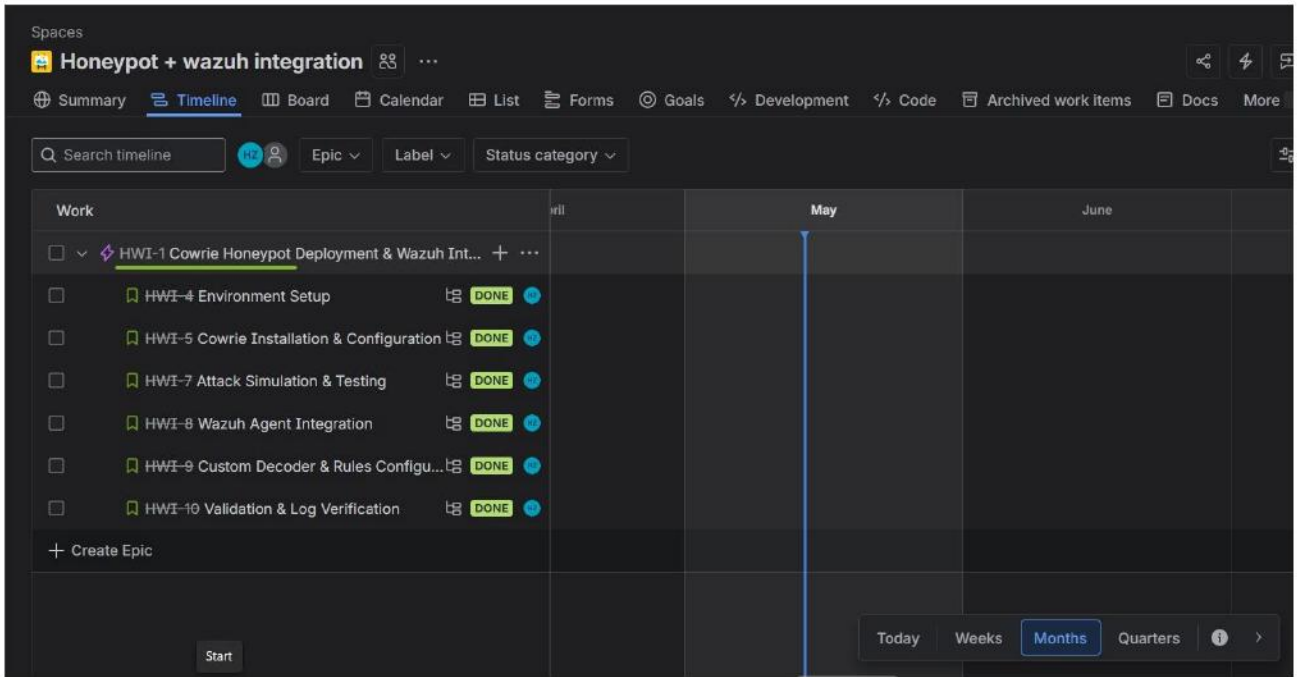


Figure 1 — Jira Timeline View: All 6 tasks marked DONE under Epic HWI-1

3.1 Epic Details

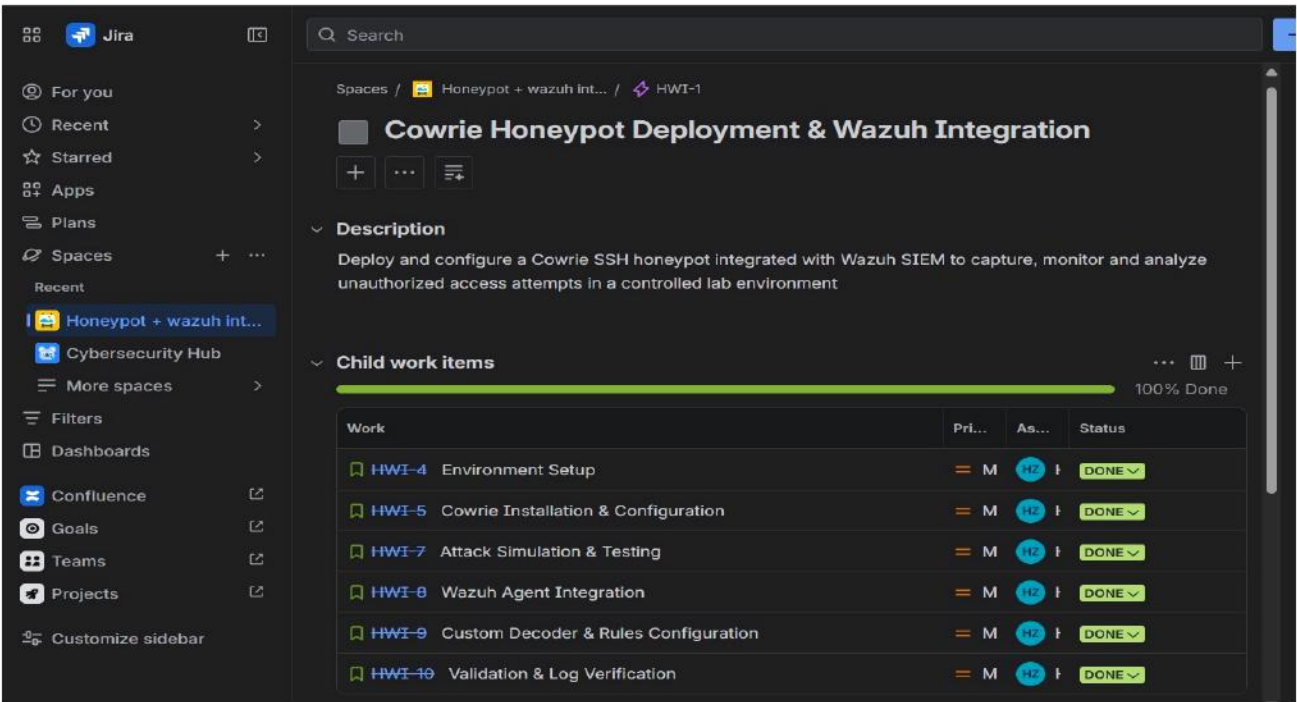


Figure 2 — Jira Epic HWI-1: Child work items showing 100% completion

The epic tracked 6 child work items, each representing a distinct phase of the project. All tasks were assigned Medium priority, and every single one reached DONE status — indicating full project completion.

Ticket ID	Task Name	Labels	Status
HWI-4	Environment Setup	setup, ssh, Honeypot	DONE
HWI-5	Cowrie Installation & Configuration	cowrie, setup	DONE
HWI-7	Attack Simulation & Testing	Attack, Nmap, Testing	DONE
HWI-8	Wazuh Agent Integration	Agent, Wazuh, Honeypot	DONE
HWI-9	Custom Decoder & Rules Configuration	cowrie, setup, decoder	DONE
HWI-10	Validation & Log Verification	Wazuh, decoder, Testing	DONE

4. TECHNICAL IMPLEMENTATION

4.1 Phase 1 — Environment Setup (HWI-4)

Before installing Cowrie, OpenSSH was installed and enabled on the Ubuntu server. This ensures the system is accessible via SSH for administration while Cowrie listens separately on port 2222.

Commands:

- **sudo apt install openssh-server**
- **sudo systemctl enable ssh**
- **sudo systemctl start ssh**
- **sudo systemctl status ssh**

4.2 Phase 2 — Cowrie Installation & Configuration (HWI-5)

Cowrie was installed from its official GitHub repository inside a dedicated Python virtual environment, running under a separate non-privileged system user for security isolation.

- **sudo apt update**
- **sudo apt upgrade**
- **sudo apt install python3 python3-venv python3-pip git libssl-dev libffi-dev build-essential libpython3-dev virtualenv -y**

```
~$ sudo apt install python3 python3-venv python3-pip git libssl-dev libffi-dev build-essential libpython3-dev virtualenv -y
```

- **sudo adduser --disabled-password cowrie**

```
~$ sudo adduser --disabled-password cowrie
```

- **sudo su -cowrie**

```
~$ sudo su - cowrie
```

- **git clone <https://github.com/cowrie/cowrie.git>**

```
~$ git clone https://github.com/cowrie/cowrie.git
```

- **cd cowrie**
- **python3 -m venv cowrie-env**
- **source cowrie-env/bin/activate**

```
~$ cd cowrie
~/cowrie$ python3 -m venv cowrie-env
~/cowrie$ source cowrie-env/bin/activate
```

- **pip install --upgrade pip**
- **pip install -r requirements.txt**

```

:~/cowrie$ pip install --upgrade pip
pip install -r requirements.txt

```

- cp etc/userdb.txt.dist etc/userdb.txt
- bin/cowrie start

```

(cowrie-env) cowrie@thunder:~/cowrie$ bin/cowrie start
Using activated Python virtual environment /home/cowrie/cowrie/cowrie-env
Starting cowrie: [twisted --umask=0022 --pidfile=/var/run/cowrie.pid --logger cowrie.python.logfile.logger cowrie ]...
/home/cowrie/cowrie/cowrie-env/lib/python3.13/site-packages/twisted/conch/ssh/transport.py:110: CryptographyDeprecationWarning: TripleDES has been moved to cryptography.hazmat.decrepit.ciphers.algorithms.TripleDES and will be removed from cryptography.hazmat.primitives.ciphers.algorithms in 48.0.0.
  b"3des-cbc": (algorithms.TripleDES, 24, modes.CBC),
/home/cowrie/cowrie/cowrie-env/lib/python3.13/site-packages/twisted/conch/ssh/transport.py:117: CryptographyDeprecationWarning: TripleDES has been moved to cryptography.hazmat.decrepit.ciphers.algorithms.TripleDES and will be removed from cryptography.hazmat.primitives.ciphers.algorithms in 48.0.0.
  b"3des-ctr": (algorithms.TripleDES, 24, modes.CTR),

```

- bin/cowrie status

```

(cowrie-env) cowrie@thunder:~/cowrie$ bin/cowrie status
cowrie is running (PID: 3885).

```

Cowrie Version 2.6.1 started successfully using Twisted 25.5.0 and Python 3.13.3, confirming a fully operational honeypot environment.

4.3 Phase 3 — Attack Simulation & Testing (HWI-7)

A Kali Linux machine was used to simulate attacker behavior. The attacker first performed a network scan to discover open ports, then attempted SSH brute-force login.

- Honeypot Server IP is **192.168.184.134**
- Kali Linux(Attacker) IP is **192.168.184.129**

```

$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.184.129 netmask 255.255.255.0 broadcast 192.168.184.255
    inet6 fe80::20c:29ff:fece:8a76 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:ce:8a:76 txqueuelen 1000 (Ethernet)
    RX packets 15369 bytes 15915830 (15.1 MiB)
    RX errors 0 dropped 16 overruns 0 frame 0
    TX packets 10488 bytes 1139077 (1.0 MiB)
    TX errors 0 dropped 5 overruns 0 carrier 0 collisions 0

```

Step 1: NMAP port scan from Kali Linux (attacker)

```

$ sudo nmap -p- -T4 192.168.184.134
Starting Nmap 7.95 ( https://nmap.org ) at 2025-07-04 13:20 EDT
Nmap scan report for 192.168.184.134 (192.168.184.134)
Host is up (0.0025s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
2222/tcp  open  EtherNetIP-1
MAC Address: 00:0C:29:4A:58:2A (VMware)

Nmap done: 1 IP address (1 host up) scanned in 10.42 seconds

```


Step 2: SSH brute-force attempt against Cowrie

```

$ ssh root@192.168.184.128
root@192.168.184.128's password:
Permission denied, please try again.
root@192.168.184.128's password:
Permission denied, please try again.
root@192.168.184.128's password:
root@192.168.184.128: Permission denied (publickey,password).

```

Cowrie intercepted the connection on port 2222, logged the attacker's IP (192.168.184.129), recorded every password attempt, and silently denied access — while making the attacker believe they were hitting a real SSH server.

LOGS:

Cowrie raw log excerpt (tail -f var/log/cowrie/cowrie.log):

```

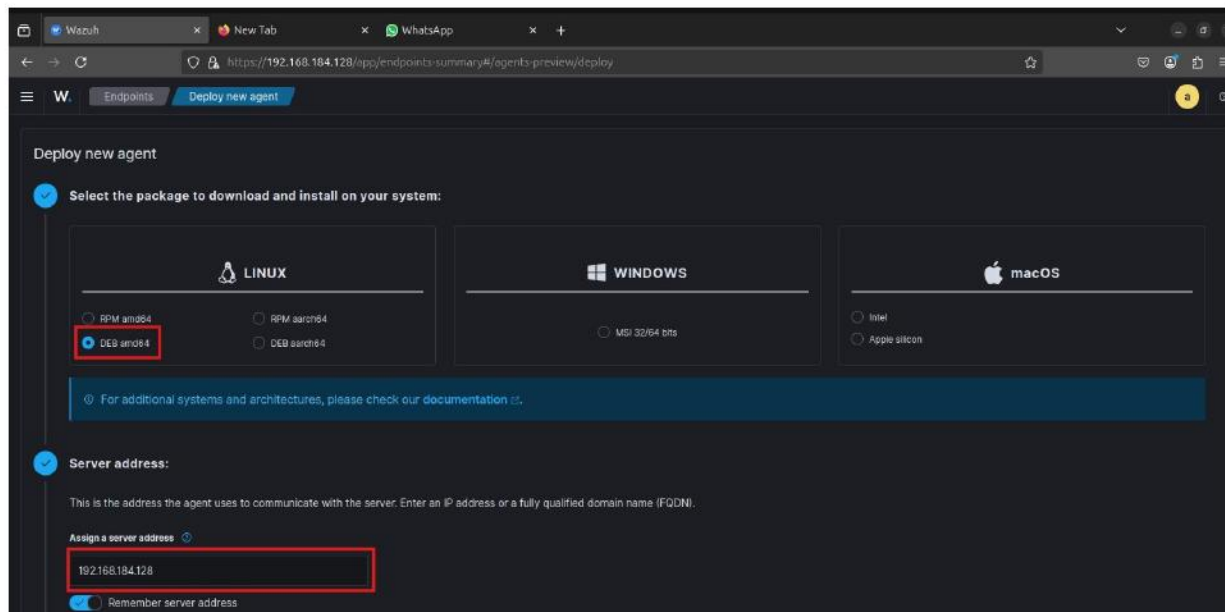
(cowrie-env) cowrie@allraza:~/cowrie$ tail -f var/log/cowrie/cowrie.log
2025-07-04T18:49:08.999153Z [-] Cowrie Version 2.6.1
2025-07-04T18:49:09.019854Z [-] Loaded output engine: jsonlog
2025-07-04T18:49:09.026868Z [twisted.scripts._twisted_unix.UnixAppLogger#info] twisted 25.5.0 (/home/cowrie/cowrie/cowrie-env/bin/python3 3.13.3) starting up.
2025-07-04T18:49:09.021127Z [twisted.scripts._twisted_unix.UnixAppLogger#info] reactor class: twisted.internet.epollreactor.EPollReactor.
2025-07-04T18:49:09.231108Z [-] CowrieSSHFactory starting on 2222
2025-07-04T18:49:09.251734Z [cowrie.ssh.factory.CowrieSSHFactory#info] Starting factory <cowrie.ssh.factory.CowrieSSHFactory object at 0x794dfc351550>
2025-07-04T18:49:09.317139Z [-] Generating new RSA keypair...
2025-07-04T18:49:09.621684Z [-] Generating new ECDSA keypair...
2025-07-04T18:49:09.666876Z [-] Generating new ed25519 keypair...
2025-07-04T18:49:09.747794Z [-] Ready to accept SSH connections
2025-07-04T18:58:35.781162Z [cowrie.ssh.factory.CowrieSSHFactory] New connection: 192.168.184.129:55524 (192.168.184.128:2222) [session: d085fbd17278]
2025-07-04T18:58:35.782310Z [HoneyPotSSHTransport,0,192.168.184.129] Remote SSH version: SSH-2.0-OpenSSH_10.0p2 Debian-5
2025-07-04T18:58:35.785519Z [HoneyPotSSHTransport,0,192.168.184.129] SSH client hassh fingerprint: eeca2460550b9ded084ecf2f70a75356
2025-07-04T18:58:35.787752Z [cowrie.ssh.transport.HoneyPotSSHTransport#debug] kex alg=b'curve25519-sha256' key alg=b'ssh-ed25519'
2025-07-04T18:58:35.788952Z [cowrie.ssh.transport.HoneyPotSSHTransport#debug] outgoing: b'aes128-ctr' b'hmac-sha2-256' b'none'
2025-07-04T18:58:35.788952Z [cowrie.ssh.transport.HoneyPotSSHTransport#debug] incoming: b'aes128-ctr' b'hmac-sha2-256' b'none'
2025-07-04T18:58:40.424973Z [cowrie.ssh.transport.HoneyPotSSHTransport#debug] NEW KEYS
2025-07-04T18:58:40.426172Z [cowrie.ssh.transport.HoneyPotSSHTransport#debug] starting service b'ssh-userauth'
2025-07-04T18:58:40.427648Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] b'root' trying auth b'none'
2025-07-04T18:58:40.435100Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] b'root' trying auth b'password'
2025-07-04T18:58:40.437293Z [HoneyPotSSHTransport,0,192.168.184.129] login attempt [b'root'/b' ] failed
2025-07-04T18:58:40.439369Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] b'root' failed auth b'password'
2025-07-04T18:58:40.439792Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] unauthorized login: ()
2025-07-04T18:58:58.554555Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] b'root' trying auth b'password'
2025-07-04T18:58:58.555123Z [HoneyPotSSHTransport,0,192.168.184.129] login attempt [b'root'/b'a ] failed
2025-07-04T18:58:59.556934Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] b'root' failed auth b'password'
2025-07-04T18:58:59.557204Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] unauthorized login: ()
2025-07-04T18:59:00.971695Z [cowrie.ssh.transport.HoneyPotSSHTransport#info] connection lost
2025-07-04T18:59:00.972885Z [HoneyPotSSHTransport,0,192.168.184.129] Connection lost after 25.2 seconds
2025-07-04T18:59:02.253992Z [cowrie.ssh.factory.CowrieSSHFactory] New connection: 192.168.184.129:40424 (192.168.184.128:2222) [session: 5bc5307da8cd]
2025-07-04T18:59:02.254974Z [HoneyPotSSHTransport,1,192.168.184.129] Remote SSH version: SSH-2.0-OpenSSH_10.0p2 Debian-5
2025-07-04T18:59:02.261214Z [HoneyPotSSHTransport,1,192.168.184.129] SSH client hassh fingerprint: eeca2460550b9ded084ecf2f70a75356
2025-07-04T18:59:02.262861Z [cowrie.ssh.transport.HoneyPotSSHTransport#debug] kex alg=b'curve25519-sha256' key alg=b'ssh-ed25519'

```

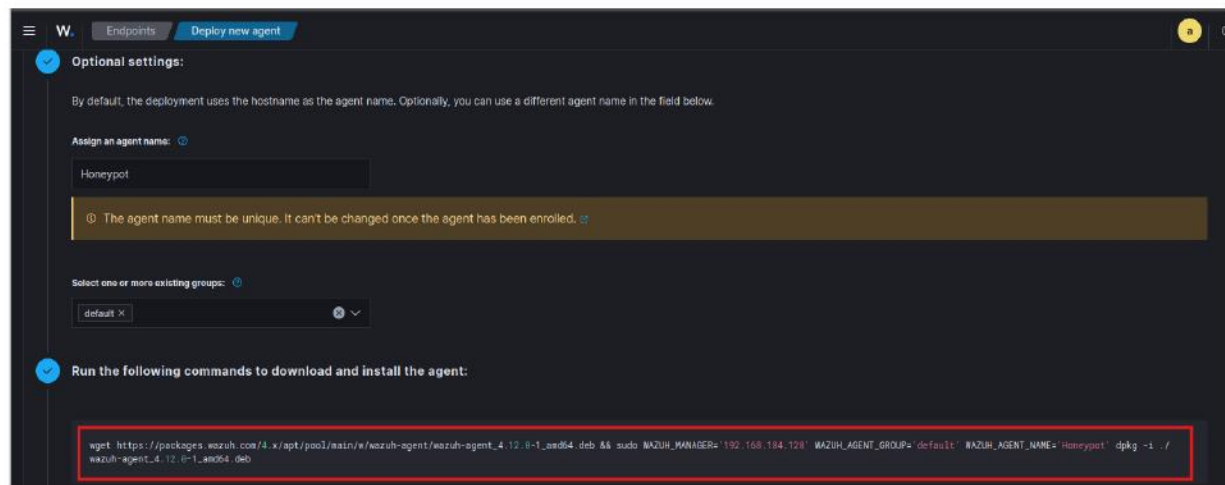

5. WAZUH INTEGRATION & ALERTS

5.1 Wazuh Agent Deployment (HWI-8)

A Wazuh agent (v4.12.0) was deployed on the honeypot server (192.168.184.134) and registered under the agent name 'Honeypot'. The agent was configured to communicate with the Wazuh manager at 192.168.184.128.



The screenshot shows the Wazuh web interface for deploying a new agent. The 'Deploy new agent' section is active. Under 'Select the package to download and install on your system:', the 'Linux' tab is selected, and 'DEB amd64' is chosen. The 'Server address' field is set to '192.168.184.128'.



The screenshot shows the 'Optional settings' section of the Wazuh web interface. The 'Assign an agent name' field is set to 'Honeypot'. The 'Run the following commands to download and install the agent:' section shows the command to download and install the agent.

Start The Agent:

```
sudo systemctl daemon-reload
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent
sudo systemctl status wazuh-agent
```

Installation of Agent: Agent IP is 192.168.184.134

```
thunder@thunder:~$ wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.12.0-1_amd64.deb && sudo
WAZUH_MANAGER='192.168.184.128' WAZUH_AGENT_GROUP='default' WAZUH_AGENT_NAME='Honeypot' dpkg -i ./wazuh-agent_4.12.0-1
_amd64.deb
--2025-07-04 22:43:25-- https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.12.0-1_amd64.deb
Resolving packages.wazuh.com (packages.wazuh.com)... 108.139.79.39, 108.139.79.29, 108.139.79.15, ...
Connecting to packages.wazuh.com (packages.wazuh.com)[108.139.79.39]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 11963008 (11M) [application/vnd.debian.binary-package]
Saving to: 'wazuh-agent_4.12.0-1_amd64.deb'

wazuh-agent_4.12.0-1_amd64.de 100%[=====] 11.41M 1.05MB/s in 10s

2025-07-04 22:43:36 (1.10 MB/s) - 'wazuh-agent_4.12.0-1_amd64.deb' saved [11963008/11963008]

[sudo] password for thunder:
Selecting previously unselected package wazuh-agent.
(Reading database ... 147959 files and directories currently installed.)
Preparing to unpack .../wazuh-agent_4.12.0-1_amd64.deb ...
Unpacking wazuh-agent (4.12.0-1) ...
Setting up wazuh-agent (4.12.0-1) ...
```

5.2 Custom Decoder & Rules (HWI-9)

To allow Wazuh to correctly parse Cowrie's JSON log format, a custom decoder was created on the Wazuh manager server:

Decoder for Cowrie:

- `sudo nano /var/ossec/etc/decoders/cowrie_decoder.xml`

```
thunder@thunder:~$ sudo nano /var/ossec/etc/decoders/cowrie_decoder.xml
```

```
decoder name="cowrie">
<program_name>cowrie</program_name>
</decoder>
```

```
GNU nano 8.3
<decoder name="cowrie">
  <program_name>cowrie</program_name>
</decoder>
```

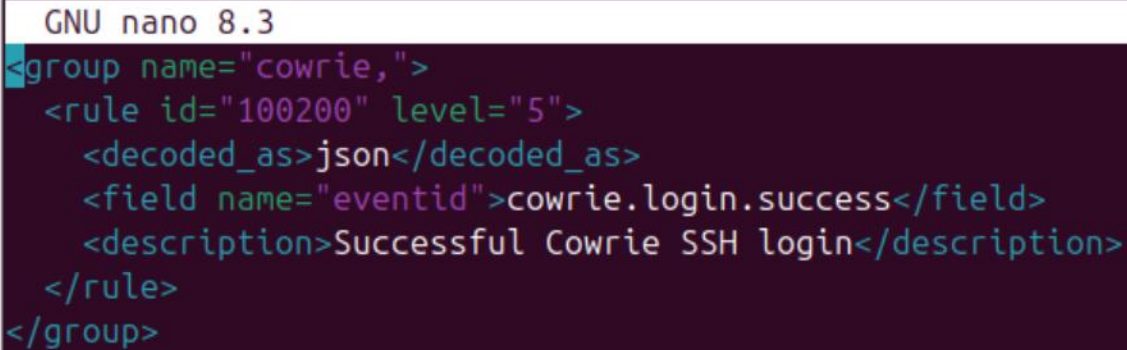
A corresponding alert rule was created to trigger on successful Cowrie SSH login events:

- `sudo nano /var/ossec/etc/rules/cowrie_rules.xml`

```
thunder@thunder:~$ sudo nano /var/ossec/etc/rules/cowrie_rules.xml
```

```
<group name="cowrie,">
  <rule id="100200" level="5">
    <decoded_as>json</decoded_as>
```

```
<field name="eventid">cowrie.login.success</field>
<description>Successful Cowrie SSH login</description>
</rule>
</group>
```



```
GNU nano 8.3
<group name="cowrie,">
  <rule id="100200" level="5">
    <decoded_as>json</decoded_as>
    <field name="eventid">cowrie.login.success</field>
    <description>Successful Cowrie SSH login</description>
  </rule>
</group>
```

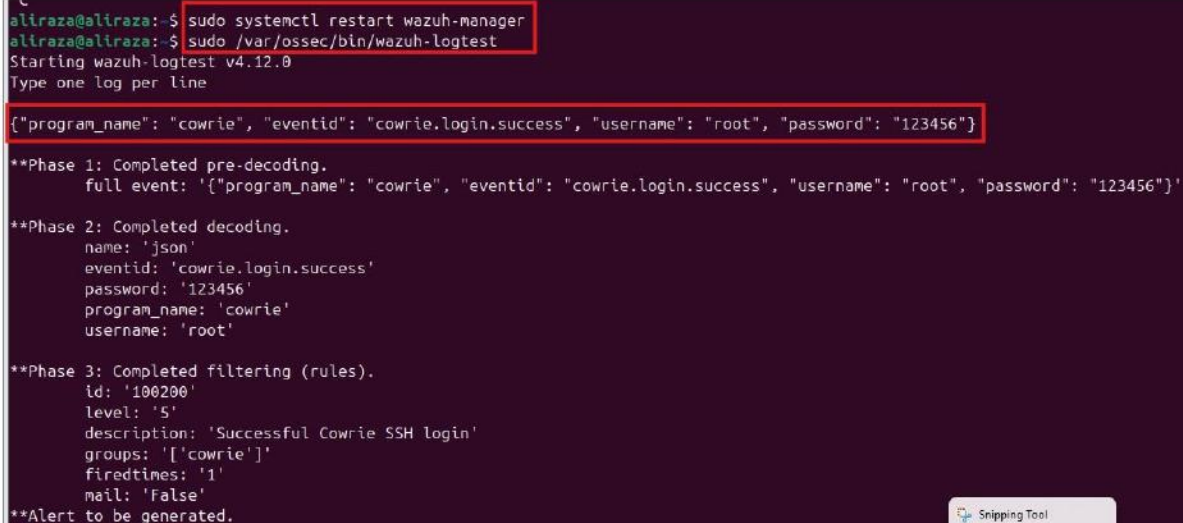
5.3 Decoder Validation (HWI-10)

Restart your wazuh-manager once

- `sudo systemctl restart wazuh-manager`
- `sudo /var/ossec/bin/wazuh-logtest`

After the second cmd enter this to verify decoder and rules are working perfectly without error

```
{"program_name": "cowrie", "eventid": "cowrie.login.success", "username": "root",
"password": "123456"}
```



```
aliraza@aliraza:~$ sudo systemctl restart wazuh-manager
aliraza@aliraza:~$ sudo /var/ossec/bin/wazuh-logtest
Starting wazuh-logtest v4.12.0
Type one log per line

{"program_name": "cowrie", "eventid": "cowrie.login.success", "username": "root", "password": "123456"}

**Phase 1: Completed pre-decoding.
  full event: '{"program_name": "cowrie", "eventid": "cowrie.login.success", "username": "root", "password": "123456"}'

**Phase 2: Completed decoding.
  name: 'json'
  eventid: 'cowrie.login.success'
  password: '123456'
  program_name: 'cowrie'
  username: 'root'

**Phase 3: Completed filtering (rules).
  id: '100200'
  level: '5'
  description: 'Successful Cowrie SSH login'
  groups: '['cowrie']'
  firetimes: '1'
  mail: 'False'

**Alert to be generated.
```

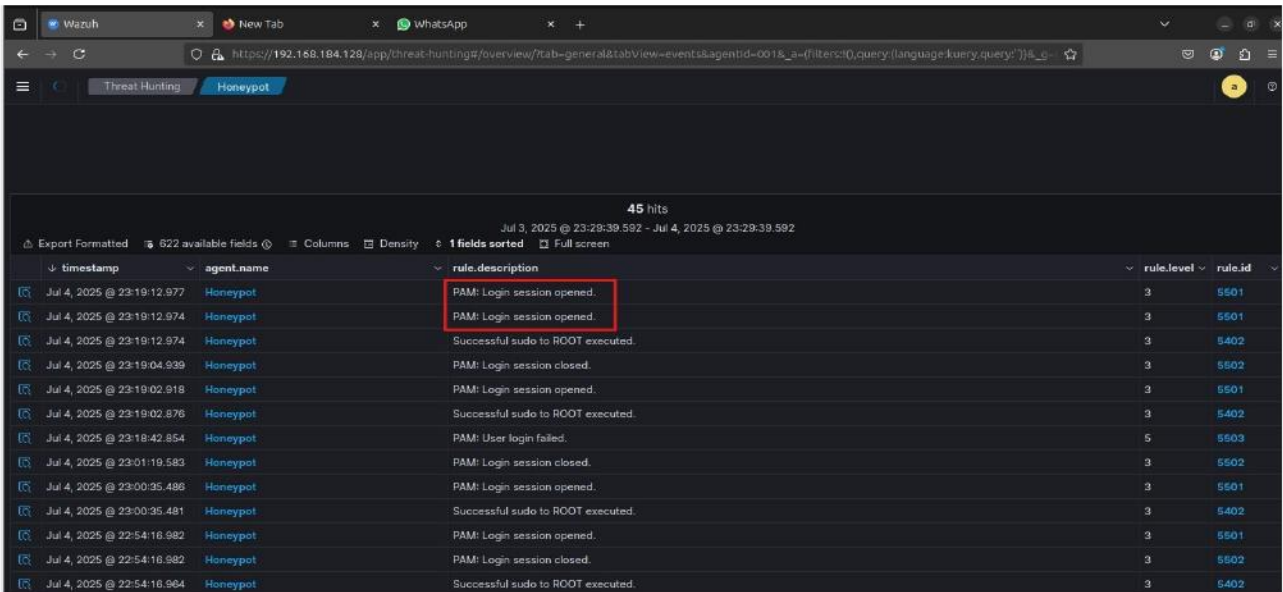

Again Test Cowrie logs:

- `tail -f var/log/cowrie/cowrie.log`

```
2025-07-04T23:26:21.140688Z [cowrie.ssh.transport.HoneyPotSSHTransport#info] connection lost
2025-07-04T23:26:21.141092Z [HoneyPotSSHTransport,3,192.168.184.129] Connection lost after 0.0 seconds
```

5.4 Live Dashboard Results

After integration, a second round of attacks was performed. The Wazuh dashboard registered 45 total hits between July 3–4, 2025. The logs revealed critical threat intelligence automatically mapped to the MITRE ATTACK framework.



The screenshot shows the Wazuh Threat Hunting dashboard with a table of 45 hits. The table has columns for timestamp, agent.name, rule.description, rule.level, and rule.id. Two rows are highlighted with red boxes, showing 'PAM: Login session opened.' events.

timestamp	agent.name	rule.description	rule.level	rule.id
Jul 4, 2025 @ 23:19:12.977	Honeypot	PAM: Login session opened.	3	5601
Jul 4, 2025 @ 23:19:12.974	Honeypot	PAM: Login session opened.	3	5601
Jul 4, 2025 @ 23:19:12.974	Honeypot	Successful sudo to ROOT executed.	3	5402
Jul 4, 2025 @ 23:19:04.939	Honeypot	PAM: Login session closed.	3	5602
Jul 4, 2025 @ 23:19:02.918	Honeypot	PAM: Login session opened.	3	5601
Jul 4, 2025 @ 23:19:02.876	Honeypot	Successful sudo to ROOT executed.	3	5402
Jul 4, 2025 @ 23:18:42.854	Honeypot	PAM: User login failed.	5	5603
Jul 4, 2025 @ 23:01:19.583	Honeypot	PAM: Login session closed.	3	5602
Jul 4, 2025 @ 23:00:35.486	Honeypot	PAM: Login session opened.	3	5601
Jul 4, 2025 @ 23:00:35.481	Honeypot	Successful sudo to ROOT executed.	3	5402
Jul 4, 2025 @ 22:54:16.982	Honeypot	PAM: Login session opened.	3	5601
Jul 4, 2025 @ 22:54:16.982	Honeypot	PAM: Login session closed.	3	5602
Jul 4, 2025 @ 22:54:16.964	Honeypot	Successful sudo to ROOT executed.	3	5402

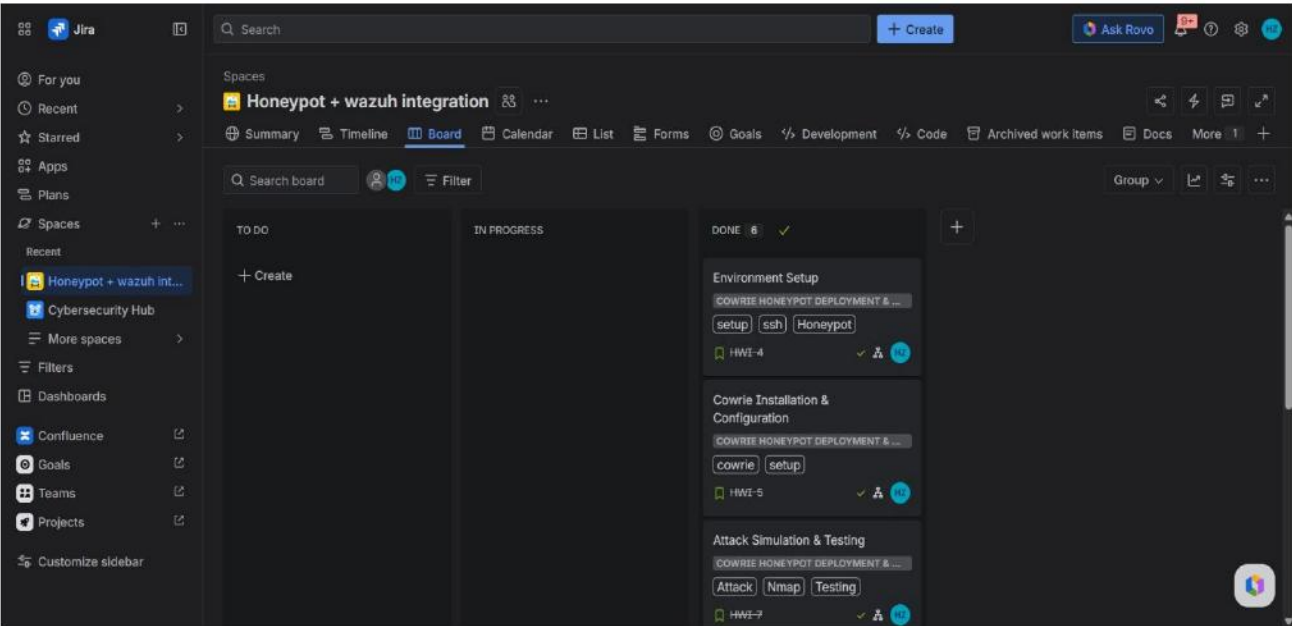


Figure 3 — Jira Board: All 6 tasks completed (DONE column)

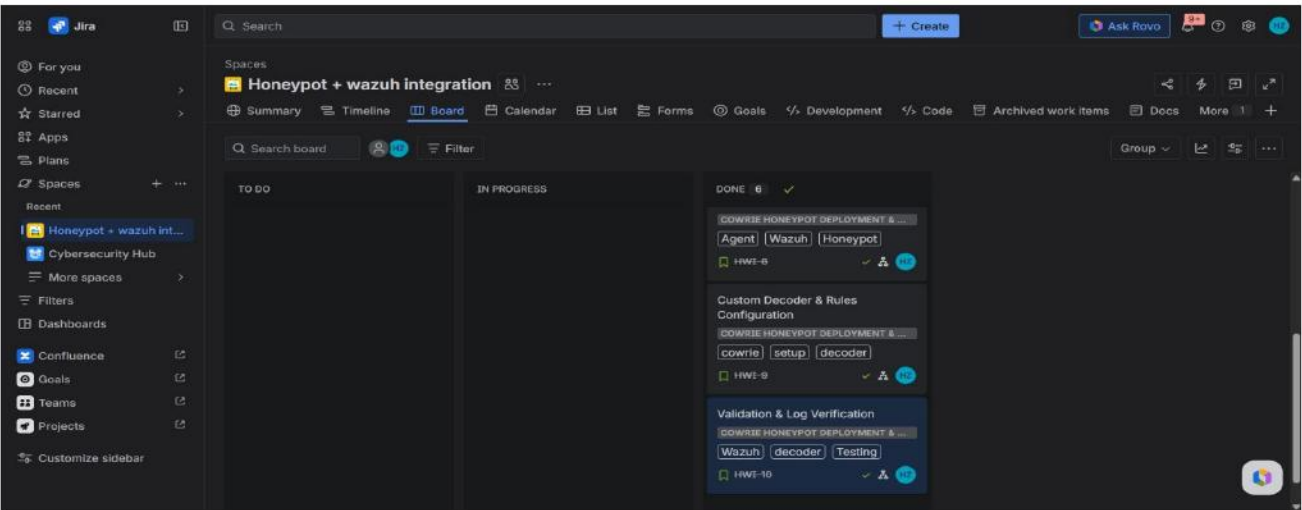


Figure 4 — Jira Board: Additional completed tasks — Custom Decoder & Validation

5.5 MITRE ATTACK Mapping

Wazuh automatically mapped the detected attack to the MITRE ATTACK framework, providing structured threat intelligence without any manual analysis:

Field	Value
MITRE ID	T1110.001
Tactic	Credential Access
Technique	Password Guessing (Brute Force)
Rule ID	5503
Rule Level	5 (Medium Severity)
Compliance	GDPR: IV_35.7.d, IV_32.2 HIPAA: 164.312.b PCI-DSS: 10.2.4, 10.2.5
Total Alerts	45 hits detected in Wazuh dashboard

Detail of log:

Document Details		View surrounding documents	View single document
Table JSON			
_index	wazuh-alerts-4.x-2025.07.04		
agent.id	001		
agent.ip	192.168.184.134		
agent.name	Honeypot		
data.dstuser	cowrie(uid=1001)		
data.srcuser	thunder		
data.uid	0		
decoder.name	pam		
decoder.parent	pam		
full_log	Jul 04 18:19:11 thunder su[12021]: pam_unix(su:session): session opened for user cowrie(uid=1001) by thunder(uid=0)		
id	1751653152.277898		
input.type	log		
location	journald		
manager.name	aliraza		
predecoder.hostname	thunder		
predecoder.program_name	su		
predecoder.timestamp	Jul 04 18:19:11		

id	1751655298.293497
input.type	log
location	journald
manager.name	aliraza
predecoder.hostname	thunder
predecoder.program_name	sudo
predecoder.timestamp	Jul 04 18:54:56
rule.description	PAM: User login failed.
rule.firedtimes	3
rule.gdpr	IV_35.7.d, IV_32.2
rule.gpg13	7.8
rule.groups	pam, syslog, authentication_failed
rule.hipaa	164.312.b
rule.id	5503
rule.level	5
rule.mail	false
rule.mitre.id	T1110.001
rule.mitre.tactic	Credential Access
rule.mitre.technique	Password Guessing
rule.nist_800_53	AU.14, AC.7
rule.pci_dss	10.2.4, 10.2.5
rule.tsc	CC6.1, CC6.8, CC7.2, CC7.3

7. CONCLUSION & LESSONS LEARNED

7.1 Summary of Achievements

This project successfully demonstrated how a Cowrie honeypot, combined with the Wazuh SIEM platform, creates a powerful and cost-effective threat detection system. The setup trapped and recorded a simulated attacker's full behavior — from initial network reconnaissance to repeated SSH brute-force attempts — without ever exposing a real system to risk.

The Wazuh integration elevated the honeypot from a passive logging tool into an active, alert-driven security monitoring system. Custom decoders and rules ensured that Cowrie's JSON logs were properly ingested, and MITRE ATT&CK tagging provided immediate, actionable threat context for every event.

7.2 Lessons Learned

- Cowrie must run under a non-privileged user account and in its own virtual environment to prevent any attacker from pivoting to the real system. Honeypot Isolation is Critical:
- Wazuh's decoder engine is strict about field names and JSON structure. Careful testing with wazuh-logtest before deployment saved significant debugging time. Decoders Require Exact Matching:
- Having attack techniques automatically mapped removes the need for manual threat classification, making the platform viable even for small security teams. MITRE ATT&CK Adds Immediate Value:
- IP address consistency across documentation (attacker: 192.168.184.129, honeypot: 192.168.184.134) is essential for accurate incident reconstruction and log correlation. Log Accuracy Matters:
- Tracking each technical task in Jira provided clear accountability, progress visibility, and a professional audit trail for the entire engagement. Jira as a Security PM Tool:

7.3 Potential Improvements

- Deploy Cowrie on a public-facing cloud instance to capture real-world internet attacks and build genuine threat intelligence.
- Add GeoIP enrichment to Wazuh to visualize attacker locations on a world map dashboard.
- Configure email or Slack alerting in Wazuh for immediate notification on high-severity events.
- Expand the Cowrie ruleset to detect malicious command execution (e.g., wget, curl, /etc/passwd reads) inside the honeypot shell.
- Integrate with MISP or OpenCTI for automated Indicator of Compromise (IoC) sharing.